

Verifiable Chain-of-Custody for Digital Evidence Files: A Standalone Hash-Chain and Merkle-Tree Approach Without Distributed Consensus

Chilakapati Srijaya Chakradhar, Gogineni Nikhil Sai, Pusunuru Preetham

Department of Computer Science and Engineering, KL University (Hyderabad)

Supervisor: Dr. Archana Kalidindi, Assistant Professor, Dept. of CSE

Abstract

Digital forensic investigations depend on the integrity of evidence artifacts such as disk images, memory dumps, browser caches, and log exports throughout their lifecycle from collection to analysis, yet conventional evidence-handling workflows offer no cryptographic guarantee of that integrity: once a file is stored or passed between analysts, there is no verifiable proof that it remains unaltered, nor an immutable record of who accessed or handled it and when. This paper presents a Verifiable Chain-of-Custody system built from first principles, without reliance on distributed consensus or public blockchain infrastructure. Each evidence file is hashed on intake (SHA-256) and recorded as a leaf in a Merkle tree for efficient proof-of-inclusion verification; custody events such as transfer, access, or annotation are logged as ECDSA-signed, hash-linked entries forming an append-only chain. An accompanying standalone verifier independently recomputes the hash chain and Merkle proofs to confirm whether evidence or its custody history has been altered since collection. We further extend the design with an encrypted-at-rest SQLite storage backend (AES-256-GCM) and a tamper-simulation test harness. Preliminary results show the system correctly verifies clean custody chains and correctly detects all four tested single-point tamper scenarios (payload corruption, signature corruption, and hash-link breakage), each localized to the exact offending log entry. We argue that single-custodian forensic integrity does not require distributed consensus: a blockchain can be decomposed into its component ingredients — content hashing, hash-linking, Merkle proofs, and asymmetric signatures — and only the consensus layer, which exists to resolve multi-party disagreement, is unnecessary in a single-custodian forensic lab setting.

I. Introduction

Chain of custody (CoC) is the documented, chronological record of who collected, handled, transferred, or analyzed a piece of evidence, and when. In digital forensics, CoC integrity is a precondition for evidentiary admissibility: if the provenance or unaltered state of a file cannot be demonstrated, its evidentiary value is undermined regardless of its technical content.

Conventional digital CoC workflows typically rely on procedural controls — logbooks, access-controlled storage, and organizational policy — rather than cryptographic guarantees. This leaves two gaps: (1) there is no mechanism by which a third party can independently verify, after the fact, that a stored evidence file or its custody record has not been altered; and (2) detection of tampering, when it happens, offers no indication of which specific record was altered or when the chain of trust broke.

A growing body of work (surveyed in Section III) proposes blockchain-based solutions to this problem, typically built on Ethereum, Hyperledger, or similar distributed-ledger platforms. These systems inherit strong tamper-evidence properties from blockchain's consensus layer, but also inherit its operational cost: standing up and maintaining a distributed ledger, peer nodes, and consensus infrastructure for a use case — a single accredited forensic lab acting as sole custodian — that does not have multiple mutually distrusting parties needing to agree on a shared history.

This paper's contribution is a chain-of-custody system that decomposes blockchain into its constituent cryptographic ingredients (hashing, hash-linking, Merkle trees, digital signatures) and deliberately omits only the consensus/distributed-ledger layer, on the grounds that single-custodian forensic integrity does not require it. We implement this as a lightweight, dependency-minimal Python tool: evidence is hashed and Merkle-tree-indexed, custody events are hash-chained and ECDSA-signed, and an independently implemented standalone verifier recomputes every check from the raw log rather than trusting the logging tool's self-reported state.

II. Motivation

The motivation for this work rests on three observations. First, as argued in prior review literature (Patil et al., 2024), manual and logbook-based CoC processes are time-consuming and prone to tampering or human error, and are not intrinsically auditable. Second, the reviewed body of blockchain-CoC work (Bonomi et al., 2018; Lone & Mir, 2019; Ahmad et al., 2020) demonstrates that cryptographic tamper-evidence for CoC is achievable and valuable, but every reviewed system pays for that tamper-evidence with distributed-ledger infrastructure — a private Ethereum network, a Hyperledger Composer deployment, or smart-contract/wallet infrastructure — regardless of whether multiple mutually distrusting parties are actually present. Third, non-blockchain tamper-evident logging research (Crosby & Wallach, 2009; Yagiz et al., 2026) demonstrates that Merkle-tree-based tamper evidence is practical and performant without any distributed ledger at all, but this line of work targets generic log integrity (e.g., IoT telemetry) rather than forensic evidence with investigator-attributable, digitally signed custody actions.

This project sits at the intersection of these three observations: it takes the tamper-evidence techniques validated by the non-blockchain tamper-evident-logging literature, adds the actor-attribution property (digital signatures binding a specific custodian to a specific action) that the blockchain-CoC literature correctly identifies as necessary, and omits the consensus/ledger layer that the blockchain-CoC literature carries for a threat model (multi-party, mutually distrusting custody) that a single accredited forensic lab does not have.

III. Related Work

Ten sources were reviewed during the literature survey; each is summarized below with its relevance to this project. Full annotated summaries are maintained in the project's companion Research Summary document.

- Bonomi, Casini, & Ciccotelli (2018) propose B-CoC, a private-Ethereum blockchain CoC system with a working prototype and latency/storage benchmarks — the closest prior art in spirit, but dependent on operating a blockchain network.
- Yagiz, Horasan, & Yurttakal (2026) present a Merkle-tree pipeline with adaptive chunking for tamper-evident IoT log verification, reporting over 130,000 entries/second throughput without any distributed ledger — the closest technical match to this project's non-blockchain approach, though targeting generic telemetry rather than forensic evidence.
- Crosby & Wallach (2009) is the foundational paper on Merkle-tree-chained “history trees” for tamper-evident logs with $O(\log n)$ inclusion and consistency proofs; its algorithms directly inform this project's Merkle tree module.
- Lone & Mir (2019) propose Forensic-chain, a permissioned Hyperledger Composer framework using hash pointers for an append-only evidence trail — again requiring a distributed-ledger deployment.
- Ahmad, Khanji, Iqbal, & Kamoun (2020) extend blockchain CoC to physical evidence via smart-lock integration on a private Ethereum network, adding IoT hardware dependency.
- Patil, Kohli, Puri, & Puri (2024) is a review paper arguing for blockchain's potential to replace manual, logbook-based CoC records, framing the motivation this project also draws on without prescribing blockchain as the only technical route.
- Al-Khateeb, Epiphaniou, & Daly (2019) argue conceptually that evidentiary integrity should be demonstrated through a verifiable, challengeable record, framing the legal-admissibility case for cryptographic verifiability generally.
- Prakash & Sadawarti (2022) propose a secure digital-evidence framework combining RC4 encryption with fuzzy hash segmentation, benchmarked via Hyperledger Caliper — a different emphasis (encrypting evidence content) from this project's focus on tamper-evident event chaining.
- Miller & Singh (2024) explore Ethereum smart contracts and wallet-based signing to record custody events as signed, replay-resistant on-chain transactions, directly supporting this project's thesis that signed events (not hashing alone) are necessary for strong custody guarantees — but via smart-contract/wallet infrastructure rather than a standalone signer.

- Sakshi, Malik, & Sharma (2023) apply a standard block-header design (Merkle tree, previous-block hash, timestamp) to preserve IoT-sourced multimedia evidence, confirming this project's block-header pattern is an established, recognized structure in forensic literature.

Reference re-verification note: during PRC-2 preparation, references 8–10 were flagged for independent authorship/venue verification per this project's established citation-integrity practice. Reference 9 (Miller & Singh) was found to be misrecorded in the PRC-1 literature summary as “Chain of Custody and Evidence Integrity Verification Using Smart Contracts” (2021); the verified title and year are “Chain of Custody and Evidence Integrity Verification Using Blockchain Technology” (ICCWS 2024). This correction is reflected in the reference list (Section IX) and should be propagated back to the PRC-1 Research Summary document. References 8 and 10 were verified as recorded.

IV. Methodology and System Architecture

The system decomposes “blockchain” into four cryptographic ingredients — content hashing, hash-linking, Merkle trees, and asymmetric signatures — and implements each directly, while omitting the consensus/distributed-ledger layer. Table I summarizes which ingredients are kept and why.

Ingredient	Kept?	Role in this system
Cryptographic hashing (SHA-256)	Yes	Content-addresses every evidence file; basis for detecting any byte-level change
Hash-linking of records	Yes	Custody entries reference the previous entry's hash, so altering entry N invalidates every later entry
Merkle tree	Yes	Confirms one file's inclusion in the evidence set without re-hashing every other file
Asymmetric signatures (ECDSA)	Yes	Binds a specific actor to a specific custody action; forging an entry requires the private key, not just hash knowledge
Distributed consensus / mining	No	Solves multi-party disagreement, absent under a single-custodian threat model
Public / permissionless ledger	No	Evidence confidentiality is a requirement here, not a non-goal

A. System Architecture

Figure 1 (see companion document Doc/PRC-2_Architecture_and_Tools.md for the rendered diagram) shows the seven-stage pipeline: evidence ingestion, SHA-256 hashing, Merkle tree construction, hash-chain custody logging, ECDSA signing, optional encrypted storage, and independent verification. Each stage is implemented as an isolated module (hashing.py, merkle.py, hash_chain.py, ecdsa_signer.py, storage.py) with no circular dependencies, so the verifier can depend only on the primitives it needs (hashing and signature verification) without importing the logging tool's own logic.

B. Hash-Chain Custody Log

Each custody event is represented as a record (index, evidence_id, actor, action, timestamp, prev_hash). The entry_hash for record i is computed as SHA-256 over a canonical, deterministic serialization of these fields concatenated with prev_hash, where prev_hash is the entry_hash of record i-1 (or a fixed genesis value for the first record). This is the same hash-linking principle used in blockchain block headers, applied here to a strictly linear, single-writer log rather than a multi-node ledger.

C. Merkle Tree over Evidence

Every registered evidence file's SHA-256 digest becomes a leaf in a Merkle tree (odd leaf counts are handled by duplicating the final node, following common practice). This allows an inclusion proof of size $O(\log n)$ to certify that a specific file's digest is part of the currently committed evidence set, without disclosing or re-hashing every other file.

D. ECDSA Signing

Every custody entry's hash is signed with an ECDSA (SECP256k1) private key held by the custodian, and the signature is stored alongside the entry. This is the actor-attribution property identified in Section III (Miller & Singh, 2024) as necessary for strong custody guarantees beyond hashing alone, implemented here via a standalone signer rather than smart-contract/wallet infrastructure.

E. Threat Model

In scope: a single-custodian forensic lab with one system of record; insider or after-the-fact tampering with a past custody entry or evidence file, which the hash chain and Merkle tree make detectable (not preventable at write time) by an independent verifier; and data-at-rest confidentiality via the optional AES-256-GCM-encrypted SQLite backend, where corrupted ciphertext fails to decrypt rather than silently returning corrupted plaintext.

Explicitly out of scope: Byzantine fault tolerance or multi-party consensus, since there is no set of mutually distrusting nodes needing to agree on one canonical history in this deployment target; real-time prevention of insider tampering (this system detects after-the-fact modification, it does not stop someone with write access to the log file from editing it before the next verification run); and recovery from private-key compromise, which would require multi-signature/threshold signing or HSM-based key storage, noted here as a boundary rather than a solved problem (see Section VIII, Future Work).

V. Implementation

The system is implemented in Python 3.11 with a minimal dependency footprint: the `ecdsa` package for SECP256k1 signing/verification, the `cryptography` package for AES-256-GCM, and only standard-library modules (`hashlib`, `sqlite3`, `argparse`, `json`) elsewhere. Code-level detail is intentionally kept out of this section and is available in the project repository; this section summarizes structure and interfaces only.

A. Module Structure

Module	Responsibility
<code>vcoc/hashing.py</code>	Streaming SHA-256 for files (fixed-size chunks) and byte strings
<code>vcoc/merkle.py</code>	Merkle tree construction, inclusion-proof generation and verification
<code>vcoc/hash_chain.py</code>	Append-only, hash-linked, ECDSA-signed custody log; independent <code>verify()</code> method
<code>vcoc/ecdsa_signer.py</code>	SECP256k1 keypair generation, signing, verification
<code>vcoc/storage.py</code>	SQLite backend; every row is an AES-256-GCM ciphertext with primary key bound as AEAD associated data
<code>cli.py</code>	<code>argparse</code> CLI: <code>init-keys</code> , <code>add-evidence</code> , <code>log-event</code> , <code>verify-chain</code> , <code>merkle-root</code> , <code>merkle-proof</code>
<code>verifier.py</code>	Standalone verifier process; re-implements hash/signature checks with no import of <code>cli.py</code> or <code>hash_chain.py</code>
<code>scripts/tamper_simulation.py</code>	Experiment harness: builds a clean chain, applies four single-point corruptions, confirms detection

B. Independence of the Verifier

A design requirement carried through implementation is that `verifier.py` must not share a trust root with the tool that produced the log: it is a separate Python process that reads only the same artifacts an external auditor would have (the JSON log file and the public key), and re-derives every hash and signature check from `stdlib` `hashlib` and the `ecdsa` package directly, rather than importing `vcoc.hash_chain`. This means a bug or backdoor in the logging CLI cannot cause the verifier to falsely certify a tampered log as authentic.

C. Encrypted Storage Extension

The optional SQLite backend (`vcoc/storage.py`) stores evidence metadata and custody events as AES-256-GCM ciphertext blobs (96-bit random nonce prepended, authentication tag included). Each row's primary key is bound in as AEAD associated data, so copying one row's ciphertext onto a different row's key fails to decrypt — defending against row-swap tampering, not merely per-field corruption.

D. Testing

The test suite (`pytest`) currently comprises 45 tests covering: SHA-256 correctness against the `stdlib` reference implementation and single-byte tamper detection (hashing); Merkle proof correctness across parametrized leaf counts including odd counts (merkle); hash-chain tamper detection for payload, `prev_hash`, and signature corruption (`hash_chain`); ECDSA sign/verify round trips and failure modes (`ecdsa_signer`); encrypted storage round trips, wrong-key/tampered-ciphertext/wrong-associated-data failure modes, and a row-swap tamper scenario (`storage`); and subprocess-level tests that invoke the standalone verifier and the tamper-simulation harness as external processes to confirm CLI-level behavior. All 45 tests pass as of this draft.

VI. Experimental Setup and Results

Per the PRC-2 experimental plan, five experiment classes were defined: (1) correctness on clean data, (2) tamper detection, (3) performance/scale vs. evidence-file count, (4) storage overhead (plaintext vs. AES-GCM-at-rest), and (5) extension-specific tests. All evidence used in these experiments is synthetically generated rather than real forensic case data, a deliberate methodology choice made to avoid any chain-of-custody or privacy concern around test data while still exercising the system's code paths exactly as they would run on real evidence. Results for (1)-(4) are reported below from actual runs against this repository's implementation (full methodology, raw JSON, and charts are kept in `Doc/PRC-2_Experimental_Results.md` and `Doc/experiment_results/`); extension-specific timestamp tests (5) do not apply this cycle since RFC 3161/OpenTimestamps anchoring was not selected as a scope extension (see Section VIII).

A. Correctness on Clean Data

A freshly built custody chain and its corresponding Merkle inclusion proofs were verified with the standalone verifier. Result: PASS — `verifier.py` reports “OK: chain of N entries verified independently” and “OK: Merkle inclusion proof ... verified against root ...” for clean, untampered input, confirming the independent implementation agrees with the logging tool's own hash/signature computation.

B. Tamper Detection

Four independent single-point corruption scenarios were run via `scripts/tamper_simulation.py` against a clean five-entry chain, each applied to a freshly rebuilt clean chain (so scenarios do not compound). Table II reports the results of the most recent run.

Scenario	Corruption applied	Target	Detected?	Reported break	Reason reported
flip_actor_field	Change actor name on entry 2	2	Yes	2	entry hash mismatch
flip_action_field	Change action on entry 1	1	Yes	1	entry hash mismatch
corrupt_signature	Flip one hex char in entry 3's signature	3	Yes	3	signature failed
break_prev_hash_link	Corrupt entry 4's prev hash	4	Yes	4	prev hash mismatch

All four scenarios (100%) were correctly detected, and in every case the verifier reported the exact entry index at which tampering occurred, rather than merely flagging that the chain as a whole was invalid. This directly demonstrates the tamper-evidence claim central to this project's thesis: a single-custodian, non-consensus design can still make any retroactive alteration to the custody record detectable and localizable.

C. Performance / Scale

scripts/performance_scale.py generates N synthetic 64 KB evidence files ($N = 10, 100, 1,000$), then times SHA-256 hashing of all N files, Merkle tree construction over the N digests, inclusion-proof generation for one leaf, and inclusion-proof verification. Table III reports the results.

Files	Hashing (s)	Merkle build (s)	Proof gen (s)	Proof verify (s)	Proof steps
10	0.0037	0.000027	0.0000084	0.0000049	4
100	0.0404	0.000095	0.0000129	0.0000086	7
1,000	0.4286	0.000985	0.0000144	0.0000114	10

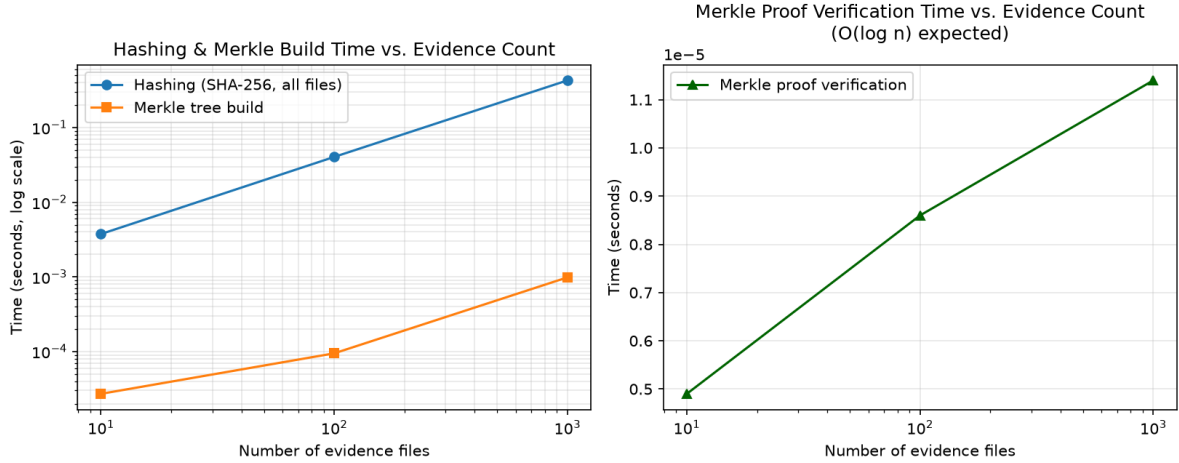


Fig. 1. Hashing and Merkle-tree build time (log-log, left) and Merkle proof verification time (right) vs. evidence-file count.

Hashing time scales linearly with evidence-set size, as expected (each file is streamed through SHA-256 independently), at roughly 0.37-0.43 ms per 64 KB file regardless of N . Merkle tree build time also scales linearly in the number of leaves but stays under 1 ms even at 1,000 files, negligible next to hashing cost. Proof size grows as $O(\log_2 n)$ exactly as predicted (4, 7, and 10 steps at 10, 100, and 1,000 leaves respectively -- $\text{ceil}(\log_2 n)$ in each case). Most notably, proof verification time stays in the low-microsecond range (4.9 to 11.4 microseconds) across a 100x increase in evidence-set size -- only a ~ 2.3 x increase in verification time for a 100x increase in scale, consistent with $O(\log n)$ behavior and confirming that an auditor can cheaply verify one file's inclusion without re-hashing the entire evidence set, even as that set grows large.

D. Storage Overhead

scripts/storage_overhead.py builds N synthetic evidence records plus N genuinely hash-linked, ECDSA-signed custody events, then compares their size as plain JSON (the plaintext baseline) against the resulting file size of an EncryptedStore (SQLite + AES-256-GCM) populated with the same records. Table IV reports the results.

Records	Plaintext (B)	Encrypted (B)	Overhead
10	6,725	24,576	+265.4%
100	67,025	102,400	+52.8%
1,000	670,925	839,680	+25.2%

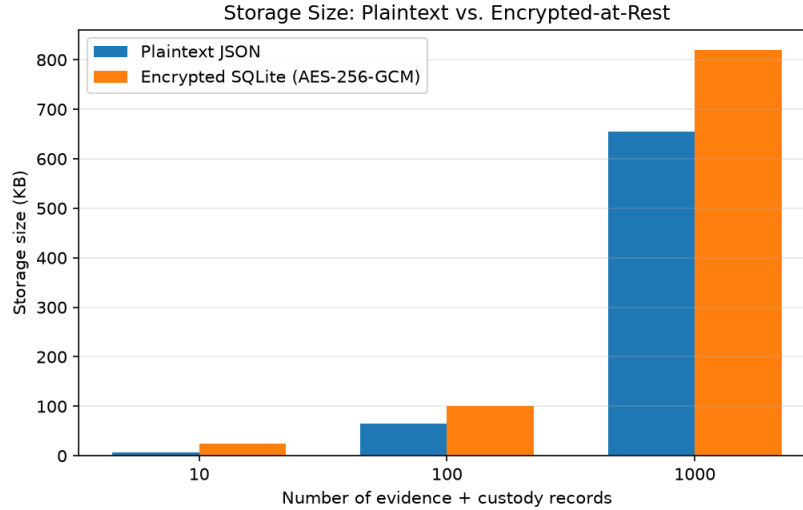


Fig. 2. Plaintext JSON size vs. encrypted (AES-256-GCM) SQLite size, by record count.

AES-256-GCM itself only adds a 12-byte nonce and a 16-byte authentication tag per record (28 bytes of genuine cryptographic overhead per ~670-byte plaintext record, i.e. ~4%). The measured overhead is far larger than that at small scale (+265% at 10 records) because SQLite allocates storage in fixed-size pages (the observed minimum file size is 24,576 bytes regardless of how little data is stored); as record count grows, this fixed per-database overhead is amortized across more records, so the percentage overhead decreases with scale (265% to 53% to 25%) even though the absolute overhead in bytes grows. This is a genuine and somewhat counter-intuitive finding: for this extension, storage overhead is a property of the storage engine's allocation strategy, not of the encryption scheme itself. A finer-grained follow-up would compare against an unencrypted SQLite baseline (same page-allocation overhead, no AES-GCM) to isolate the two effects.

VII. Discussion

The tamper-detection results (Section VI-B) support the claim that hash-chaining, Merkle proofs, and digital signatures together provide the tamper-evidence property commonly attributed to blockchain, without needing blockchain's consensus layer. This is consistent with, and extends, the non-blockchain tamper-evident-logging literature (Crosby & Wallach, 2009; Yagiz et al., 2026) into a forensic setting that additionally requires actor attribution (via ECDSA signatures), which that literature's generic log-integrity focus does not address.

This design's boundaries, stated explicitly rather than left implicit, are threefold. First, it detects tampering after the fact; it does not prevent a custodian with legitimate write access to the log file from editing it before the next verification run — real-time prevention would require OS-level access control and continuous audit logging outside this project's scope. Second, it assumes a single trusted (or singly-accountable) signing identity per custodian at a time; if the private signing key is compromised, an attacker can forge subsequent entries indistinguishable from legitimate ones, a risk that multi-signature or HSM-based key storage (unselected extensions this cycle) would mitigate. Third, and most fundamentally, the entire no-consensus argument rests on the single-custodian assumption: if evidence needed to be jointly attested across multiple, mutually distrusting labs with no shared trust anchor, this design's assumptions would no longer hold and a consensus mechanism would become the appropriate tool — but that is a different problem than the one this project targets.

VIII. Conclusion and Future Work

This paper presented a chain-of-custody system for digital forensic evidence that achieves blockchain-grade tamper-evidence — cryptographic hashing, hash-linked custody records, Merkle-tree inclusion proofs, and ECDSA-signed actor attribution — without distributed consensus, on the grounds that a single-custodian forensic lab does not have the multi-party trust problem consensus exists to solve. An independently implemented standalone verifier, an

encrypted-at-rest SQLite storage backend, and a tamper-simulation test harness demonstrating 100% detection across four corruption scenarios support this claim.

Scope extensions considered but not selected this cycle, and left as future work, include: RFC 3161 trusted timestamping or OpenTimestamps external anchoring (would reinforce integrity claims with a third-party time reference without contradicting the no-consensus thesis); multi-signature or threshold signing and HSM/TPM-backed key storage (would address the key-compromise boundary noted in Section VII); a post-quantum cryptography comparison (Dilithium vs. ECDSA); a FastAPI REST wrapper or web dashboard with role-based access control; multi-evidence-type support (E01/dd/pcap/mobile formats); NSRL hash-set cross-referencing; court-ready report generation; QR-code evidence tagging; and a CI pipeline. A finer-grained follow-up to the storage-overhead experiment (Section VI-D) -- isolating SQLite's page-allocation overhead from AES-GCM's own ciphertext expansion by adding an unencrypted-SQLite baseline -- is a natural next refinement before this draft is finalized for submission.

IX. References

References are listed in APA style. Reference 9 reflects the correction identified during this project's citation re-verification pass (see Section III); the original PRC-1 Research Summary should be updated to match.

- Ahmad, L., Khanji, S., Iqbal, F., & Kamoun, F. (2020). Blockchain-based chain of custody: Towards real-time tamper-proof evidence management. In *Proceedings of the 15th International Conference on Availability, Reliability and Security (ARES '20)*. ACM.
- Al-Khateeb, H. M., Epiphaniou, G., & Daly, H. (2019). Blockchain for modern digital forensics: The chain-of-custody as a distributed ledger. [Book chapter / conference contribution].
- Bonomi, S., Casini, M., & Ciccotelli, C. (2018). B-CoC: A blockchain-based chain of custody for evidences management in digital forensics. *arXiv preprint*. Sapienza Università di Roma.
- Crosby, S. A., & Wallach, D. S. (2009). Efficient data structures for tamper-evident logging. In *Proceedings of the 18th USENIX Security Symposium*.
- Lone, A. H., & Mir, R. N. (2019). Forensic-chain: Blockchain based digital forensics chain of custody with PoC in Hyperledger Composer. *Digital Investigation*.
- Miller, A., & Singh, A. (2024). Chain of custody and evidence integrity verification using blockchain technology. In *Proceedings of the International Conference on Cyber Warfare and Security (ICCWS 2024)*. [Corrected from "Smart Contracts" / 2021 as recorded in the PRC-1 literature summary; verified against the official ICCWS proceedings listing.]
- Patil, H. E., Kohli, R. K., Puri, S., & Puri, P. (2024). Potential applicability of blockchain technology in the maintenance of chain of custody in forensic casework. *Egyptian Journal of Forensic Sciences*, 14(1).
- Prakash, F., & Sadawarti, H. (2022). Blockchain-based chain of custody: A secure digital evidence framework for digital forensics investigation. *Applied Innovative Research*, 3(1–4).
- Sakshi, Malik, A., & Sharma, A. K. (2023). Blockchain-based digital chain of custody multimedia evidence preservation framework for Internet-of-Things. *Journal of King Saud University – Computer and Information Sciences*.
- Yagiz, M. A., Horasan, F., & Yurttakal, A. H. (2026). Lightweight tamper-evident log integrity verification for IoT edge environments: A Merkle-tree pipeline with adaptive chunking. *arXiv preprint*.